



Wallet Authentication Root Cause Analysis

Generated: October 18, 2025 - 16:45 UTC

Status: ✗ **ROOT CAUSE IDENTIFIED**

Severity: HIGH - Blocking wallet authentication



Executive Summary

The wallet authentication failure has been **definitively identified**. The frontend wallet authentication flow is calling a **missing API endpoint** (`/api/blockchain/connect`) that doesn't exist in the code-base. This is preventing users from completing the MetaMask authentication process.



Root Cause

Missing API Endpoint: `/api/blockchain/connect`

Location: Should be at `/home/ubuntu/authichain_premium/app/app/api/blockchain/connect/route.ts`

Status: ✗ **DOES NOT EXIST**

Impact:

- Frontend wallet authentication page calls this endpoint
- Endpoint is expected to verify signature and create/update user
- Without this endpoint, authentication flow fails at Step 1 (before even reaching NextAuth)



Current Authentication Flow (BROKEN)

Frontend Flow (`/app/auth/wallet/page.tsx`)

```
Step 1: User clicks "Connect MetaMask"
  ↓
Step 2: MetaMask requests account access ✓ WORKS
  ↓
Step 3: User signs authentication message ✓ WORKS
  ↓
Step 4: Frontend calls /api/blockchain/connect ✗ FAILS (404 - Endpoint doesn't exist)
  ↓
Step 5: [NEVER REACHED] NextAuth signIn('wallet', ...)
  ↓
Step 6: [NEVER REACHED] Redirect to dashboard
```

Error Point: Step 4 - The `/api/blockchain/connect` endpoint returns 404 or error

✓ What's Working

1. Frontend Wallet Integration ✓

- **File:** `/app/auth/wallet/page.tsx`
- **Status:** Fully functional
- **Features:**
 - MetaMask detection
 - Wallet connection
 - Message signing
 - Error handling
 - UI/UX components

2. Wallet Authentication Logic ✓

- **File:** `/app/lib/wallet-auth.ts`
- **Status:** Complete and correct
- **Functions Available:**

```
typescript
✓ verifyWalletSignature() - Validates signature using ethers.js
✓ createOrUpdateWalletUser() - Creates/updates user in database
✓ validateAuthMessage() - Validates message format and timestamp
```

3. NextAuth Wallet Provider ✓

- **File:** `/app/lib/auth-options.ts`
- **Status:** Configured correctly
- **Provider:** 'wallet' provider exists and ready to use
- **Logic:** Handles wallet-based authentication after signature verification

4. Dependencies ✓

- **ethers.js:** ✓ Installed and imported
- **bcryptjs:** ✓ Available for password hashing
- **Prisma:** ✓ Database client ready
- **NextAuth:** ✓ Configured properly

✗ What's Missing

1. API Endpoint: `/api/blockchain/connect`

Expected Location: `/app/app/api/blockchain/connect/route.ts`

Current Status: ✗ Does not exist

Expected Request:

```
{
  "walletAddress": "0x123...",
  "walletType": "MetaMask",
  "blockchain": "ethereum",
  "signature": "0xabc...",
  "message": "Sign this message to authenticate..."
}
```

Expected Response (Success):

```
{
  "success": true,
  "user": {
    "id": "user_id",
    "email": "0x123...@wallet.authichain.local",
    "walletAddress": "0x123...",
    "isNew": true❌false
  }
}
```

Expected Response (Error):

```
{
  "success": false,
  "error": "Invalid signature" || "User creation failed" || etc.
}
```



Solution: Create Missing API Endpoint

File to Create: /app/app/api/blockchain/connect/route.ts

Required Implementation:

```

import { NextRequest, NextResponse } from 'next/server';
import { verifyWalletSignature, createOrUpdateWalletUser, validateAuthMessage } from '@lib/wallet-auth';

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const { walletAddress, signature, message } = body;

    // Validate required fields
    if (!walletAddress || !signature || !message) {
      return NextResponse.json(
        { success: false, error: 'Missing required fields' },
        { status: 400 }
      );
    }

    // Validate message format and timestamp
    if (!validateAuthMessage(message, walletAddress)) {
      return NextResponse.json(
        { success: false, error: 'Invalid or expired message' },
        { status: 400 }
      );
    }

    // Verify signature
    const isValidSignature = await verifyWalletSignature(
      message,
      signature,
      walletAddress
    );

    if (!isValidSignature) {
      return NextResponse.json(
        { success: false, error: 'Invalid signature' },
        { status: 401 }
      );
    }

    // Create or update user
    const user = await createOrUpdateWalletUser(walletAddress);

    return NextResponse.json({
      success: true,
      user: {
        id: user.id,
        email: user.email,
        walletAddress: user.walletAddress,
        isNew: user.createdAt.getTime() === user.updatedAt.getTime()
      }
    });
  } catch (error) {
    console.error('❌ Blockchain connect error:', error);
    return NextResponse.json(
      { success: false, error: 'Internal server error' },
      { status: 500 }
    );
  }
}

```

Updated Authentication Flow (FIXED)

Complete Flow After Fix

```

Step 1: User clicks "Connect MetaMask"
↓
Step 2: MetaMask requests account access ✓
↓
Step 3: User signs authentication message ✓
↓
Step 4: Frontend calls /api/blockchain/connect ✓ (NEW - Will be created)
  → Verifies signature using ethers.js
  → Creates/updates user in database
  → Returns success response
↓
Step 5: Frontend calls NextAuth signIn('wallet', ...) ✓
  → NextAuth validates credentials
  → Creates JWT session
  → Returns session token
↓
Step 6: Redirect to dashboard ✓
  → User is now authenticated
  → Wallet address stored in session

```

Implementation Steps

Step 1: Create API Directory

```
mkdir -p /home/ubuntu/authichain_premium/app/app/api/blockchain/connect
```

Step 2: Create Route File

Create `/app/app/api/blockchain/connect/route.ts` with the implementation above

Step 3: Test the Endpoint

```

# Start dev server (if not already running)
cd /home/ubuntu/authichain_premium/app
npm run dev

# Test with curl
curl -X POST http://localhost:3000/api/blockchain/connect \
  -H "Content-Type: application/json" \
  -d '{
    "walletAddress": "0xb04d9d29f826d4837c02523ef3b50a7fbdf1d70093",
    "signature": "test_signature",
    "message": "Sign this message to authenticate with AuthiChain.\n\nWallet:
0xb04d9d29f826d4837c02523ef3b50a7fbdf1d70093\nTimestamp: '$(date +%s)000'"
  }'

```

Step 4: Test Full Wallet Authentication

1. Navigate to `http://localhost:3000/auth/wallet`

2. Click "Connect MetaMask"
3. Approve connection in MetaMask
4. Sign authentication message
5. Verify successful redirect to dashboard

Step 5: Verify Database

```
# Check that wallet user was created
cd /home/ubuntu/authichain_premium/app
npx prisma studio

# Look for user with walletAddress matching the test wallet
```

Success Criteria

After Implementation

Test	Expected Result	Status
API endpoint exists	<code>/api/blockchain/connect</code> responds	 Pending
Signature verification	Valid signatures accepted	 Pending
User creation	New wallet users created in DB	 Pending
Existing user login	Existing wallet users can log in	 Pending
Invalid signature	Rejected with proper error	 Pending
Expired message	Rejected with proper error	 Pending
NextAuth integration	Wallet provider works correctly	 Pending
Full flow	Complete authentication to dashboard	 Pending

Security Considerations

Implemented Security Features

-  Signature verification using ethers.js
-  Message timestamp validation (5-minute expiry)
-  Wallet address case-insensitive comparison

-  Protection against replay attacks (timestamp check)
-  Secure random password for wallet users (not exposed)
-  Database-level wallet address uniqueness

Additional Security Recommendations

-  Add rate limiting to prevent brute force
-  Implement nonce-based authentication for stronger security
-  Add IP-based throttling for repeated failures
-  Log all authentication attempts for audit
-  Add CAPTCHA for suspicious activity



Estimated Impact

Development Time

- **Creating endpoint:** ~15 minutes
- **Testing:** ~10 minutes
- **Documentation:** ~5 minutes
- **Total:** ~30 minutes

User Impact (After Fix)

-  Full wallet authentication functionality
-  Seamless MetaMask integration
-  One-click login for wallet users
-  No need for email/password
-  Enhanced security via signature verification



Conclusion

The wallet authentication issue is **NOT a bug** but a **missing implementation**. All the supporting infrastructure exists:

-  Frontend UI complete
-  Wallet verification logic implemented
-  NextAuth provider configured
-  Database schema supports wallet addresses

Only missing: The `/api/blockchain/connect` endpoint that ties it all together.

Fix complexity: LOW - Simple routing and wiring of existing functions

Fix time: 30 minutes

Testing time: 15 minutes

Total to deployment: ~45 minutes

Analysis Completed By: DeepAgent AI

Platform: AuthiChain Premium

Next Action: Create the missing API endpoint

Priority: HIGH ⚠️

📞 Quick Reference

Files Involved

```
✓ /app/auth/wallet/page.tsx - Frontend (complete)
✓ /app/lib/wallet-auth.ts - Logic (complete)
✓ /app/lib/auth-options.ts - NextAuth config (complete)
✗ /app/app/api/blockchain/connect/route.ts - MISSING (needs creation)
```

Command to Create Directory

```
mkdir -p /home/ubuntu/authichain_premium/app/app/api/blockchain/connect
```

Test Wallet Address

```
Address: 0xb04d9d29f826d4837c02523ef3b50a7fbdf1d70093
Private Key: 83bb9fd9e679d194ec2c207ec9fca913101c99a0eaba5aae06c9a925105e8b44
Network: Ethereum Mainnet
```

For the complete platform status, see [WALLET_AUTH_SUMMARY.md](#) (./WALLET_AUTH_SUMMARY.md)